LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. 7
Student Name K. Anusha
Roll No. A24126552088
Date 11-08-2026

1. TITLE

Perform CRUD Operations in MongoDB

2. AIM

To create a database and a collection in MongoDB and to perform the Create, Read, Update and Delete (CRUD) operations on the documents of the collection using MongoDB shell commands.

3. OBJECTIVE

The objectives of this experiment are:

- To understand databases, collections and documents in MongoDB.
- To create a database and insert multiple documents using insertMany().
- To read documents using find() and findOne() with filters, projection and sorting.
- To use comparison operators such as $gt and $lt.
- To update documents using updateOne() and updateMany() with the $set operator.
- To delete documents using deleteOne() and deleteMany().
- To verify the result of each operation.

4. THEORY

MongoDB is a NoSQL document database. Data is stored as documents in a JSON-like format called BSON, and documents are grouped into collections. A collection is similar to a table, and a document is similar to a row, but documents in the same collection can have different fields.

Create: insertOne() and insertMany() add new documents to a collection. MongoDB adds a unique _id field to every document automatically.

Read: find() returns the documents that match a filter, and findOne() returns the first match. A projection chooses which fields to show (1 to include, 0 to exclude), and sort() orders the result (1 for ascending, -1 for descending).

Update: updateOne() and updateMany() change matching documents. The $set operator changes an existing field or adds a new one.

Delete: deleteOne() removes the first matching document and deleteMany() removes all matching documents.

Query operators: $gt means greater than and $lt means less than.

5. ALGORITHM / PROCEDURE

- Start MongoDB and open the MongoDB shell (mongosh) or a MongoDB playground.
- Select the database collegeDB using use("collegeDB").
- Drop the students collection if it already exists, so the experiment starts clean.
- Insert four student documents using insertMany().
- Display all documents using find().
- Display one student using findOne() with rollNo 101.
- Display the CSE students using a filter on branch.
- Display students with marks greater than 80 using $gt.
- Display only the name and marks fields using a projection.
- Display all students sorted by marks in descending order.
- Update the marks of roll number 101 to 90 using updateOne().
- Add the field section with value "A" to all CSE students using updateMany().
- Delete the student with roll number 104 using deleteOne().
- Delete all students with marks less than 70 using deleteMany().
- Display all documents again and verify the final result.

6. PROGRAM

students.js

```
use("collegeDB");


db.students.drop();


db.students.insertMany([
  { rollNo: 101, name: "Ravi", branch: "CSE", marks: 85 },
  { rollNo: 102, name: "Asha", branch: "ECE", marks: 92 },
  { rollNo: 103, name: "Meera", branch: "CSE", marks: 78 },
  { rollNo: 104, name: "Kiran", branch: "IT", marks: 66 },
]);


db.students.find();
db.students.findOne({ rollNo: 101 });
db.students.find({ branch: "CSE" });
db.students.find({ marks: { $gt: 80 } });
db.students.find({}, { _id: 0, name: 1, marks: 1 });
db.students.find().sort({ marks: -1 });


db.students.updateOne(
  { rollNo: 101 },
  { $set: { marks: 90 } }
);

db.students.updateMany(
  { branch: "CSE" },
  { $set: { section: "A" } }
);
```

```
db.students.deleteOne({ rollNo: 104 });
db.students.deleteMany({ marks: { $lt: 70 } });


db.students.find();
```

7. CODE EXPLANATION

Code	Explanation
use("collegeDB")	Selects the database collegeDB (it is created when data is first stored).
db.students.drop()	Removes the students collection if it already exists.
insertMany([...])	Inserts four student documents into the collection.
find()	Returns all documents in the collection.
findOne({ rollNo: 101 })	Returns the first document with rollNo 101.
find({ branch: "CSE" })	Returns the documents whose branch is CSE.
{ marks: { $gt: 80 } }	Filter that matches documents with marks greater than 80.
{ _id: 0, name: 1, marks: 1 }	Projection that hides _id and shows only name and marks.
sort({ marks: -1 })	Sorts the result by marks in descending order.
updateOne(filter, { $set })	Changes the first matching document (marks of roll number 101 becomes 90).
updateMany(filter, { $set })	Adds the field section: "A" to all CSE documents.
deleteOne({ rollNo: 104 })	Deletes the document with rollNo 104.
deleteMany({ marks: { $lt: 70 } })	Deletes all documents with marks less than 70.

8. TEST CASES AND EXECUTION DATA

Each command was executed and the actual output was recorded against the expected output.

Test Case	Operation	Command	Expected Result	Actual Result	Status
TC-01	Insert	insertMany()	4 documents inserted	acknowledged: true, 4 ids returned	Pass
TC-02	Read all	find()	All 4 students displayed	4 documents displayed	Pass
TC-03	Read one	findOne({ rollNo: 101 })	Ravi displayed	Document of Ravi displayed	Pass
TC-04	Filter	find({ branch: "CSE" })	Ravi and Meera	2 documents displayed	Pass
TC-05	Operator	find({ marks: { $gt: 80 } })	Ravi (85) and Asha (92)	2 documents displayed	Pass
TC-06	Projection	find({}, { _id: 0, name: 1, marks: 1 })	Only name and marks shown	name and marks of 4 students	Pass
TC-07	Sort	sort({ marks: -1 })	Order 92, 85, 78, 66	Asha, Ravi, Meera, Kiran	Pass

Test Case	Operation	Command	Expected Result	Actual Result	Status
TC-08	Update	updateOne() on rollNo 101	Marks changed to 90	matchedCount 1, modifiedCount 1	Pass
TC-09	Update many	updateMany() on CSE	section added to 2 students	matchedCount 2, modifiedCount 2	Pass
TC-10	Delete	deleteOne({ rollNo: 104 })	Kiran removed	deletedCount 1	Pass
TC-11	Delete many	deleteMany({ marks: { $lt: 70 } })	No student below 70 remains	deletedCount 0 (Kiran was already deleted)	Pass
TC-12	Final read	find()	3 students with updates	Ravi, Asha and Meera displayed	Pass

9. OUTPUT

The output of each command in the MongoDB shell is given below. The _id values are generated automatically by MongoDB, so they are shown as ObjectId('...') and will be different on every system.

Output 1 (TC-01): insertMany()

```
> db.students.drop()
true
> db.students.insertMany([...])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...'),
    '2': ObjectId('...'),
    '3': ObjectId('...')
  }
}
```

Output 2 (TC-02): find()

```
> db.students.find()
{
  _id: ObjectId('...'),
  rollNo: 101,
  name: 'Ravi',
  branch: 'CSE',
  marks: 85
}
{
  _id: ObjectId('...'),
  rollNo: 102,
  name: 'Asha',
  branch: 'ECE',
  marks: 92
}
{
  _id: ObjectId('...'),
  rollNo: 103,
  name: 'Meera',
  branch: 'CSE',
  marks: 78
}
{
  _id: ObjectId('...'),
```

```
  rollNo: 104,
  name: 'Kiran',
  branch: 'IT',
  marks: 66
}
```

Output 3 (TC-03): findOne()

```
> db.students.findOne({ rollNo: 101 })
{
  _id: ObjectId('...'),
  rollNo: 101,
  name: 'Ravi',
  branch: 'CSE',
  marks: 85
}
```

Output 4 (TC-04): find by branch

```
> db.students.find({ branch: "CSE" })
{
  _id: ObjectId('...'),
  rollNo: 101,
  name: 'Ravi',
  branch: 'CSE',
  marks: 85
}
{
  _id: ObjectId('...'),
  rollNo: 103,
  name: 'Meera',
  branch: 'CSE',
  marks: 78
}
```

Output 5 (TC-05): find with $gt

```
> db.students.find({ marks: { $gt: 80 } })
{
  _id: ObjectId('...'),
  rollNo: 101,
  name: 'Ravi',
  branch: 'CSE',
  marks: 85
}
{
  _id: ObjectId('...'),
  rollNo: 102,
  name: 'Asha',
  branch: 'ECE',
  marks: 92
}
```

Output 6 (TC-06): projection

```
> db.students.find({}, { _id: 0, name: 1, marks: 1 })
{ name: 'Ravi', marks: 85 }
{ name: 'Asha', marks: 92 }
{ name: 'Meera', marks: 78 }
{ name: 'Kiran', marks: 66 }
```

Output 7 (TC-07): sort by marks

```
> db.students.find().sort({ marks: -1 })
{
  _id: ObjectId('...'),
  rollNo: 102,
  name: 'Asha',
  branch: 'ECE',
  marks: 92
}
{
  _id: ObjectId('...'),
  rollNo: 101,
  name: 'Ravi',
  branch: 'CSE',
  marks: 85
}
{
  _id: ObjectId('...'),
  rollNo: 103,
  name: 'Meera',
  branch: 'CSE',
  marks: 78
}
{
  _id: ObjectId('...'),
  rollNo: 104,
  name: 'Kiran',
  branch: 'IT',
  marks: 66
}
```

Output 8 (TC-08): updateOne()

```
> db.students.updateOne({ rollNo: 101 }, { $set: { marks: 90 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Output 9 (TC-09): updateMany()

```
> db.students.updateMany({ branch: "CSE" }, { $set: { section: "A" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

Output 10 (TC-10): deleteOne()

```
> db.students.deleteOne({ rollNo: 104 })
{
  acknowledged: true,
  deletedCount: 1
}
```

Output 11 (TC-11): deleteMany()

```
> db.students.deleteMany({ marks: { $lt: 70 } })
{
  acknowledged: true,
  deletedCount: 0
}
```

Output 12 (TC-12): final find()

```
> db.students.find()
{
  _id: ObjectId('...'),
  rollNo: 101,
  name: 'Ravi',
  branch: 'CSE',
  marks: 90,
  section: 'A'
}
{
  _id: ObjectId('...'),
  rollNo: 102,
  name: 'Asha',
  branch: 'ECE',
  marks: 92
}
{
  _id: ObjectId('...'),
  rollNo: 103,
  name: 'Meera',
  branch: 'CSE',
  marks: 78,
  section: 'A'
}
```

10. RESULT

Thus, the CRUD operations were successfully performed in MongoDB. Student documents were inserted with insertMany(), read using find() and findOne() with filters, projection and sorting, updated using updateOne() and updateMany(), and deleted using deleteOne() and deleteMany(). The final find() confirmed the expected result, and all test cases passed.